Resolución del Práctico 3: Compactadores

Ejercicio 1

a. Clasificación de los códigos

- **Código a:** Bloque (k = 3 bits), no singular, instantáneo y unívocamente decodificable (UD).
- **Código b:** No es de bloque, no singular, no instantáneo ('0' es prefijo de '01') y UD (el '0' inicial funciona como delimitador de inicio).
- **Código c:** No es de bloque, no singular, instantáneo (libre de prefijos) y UD.
- **Código d:** No es de bloque, no singular, no instantáneo (s2 '10' es prefijo de s5 '1011') y no UD (por Sardinas-Patterson, la secuencia '11010' admite dos decodificaciones: s3 s2 o s6 s1).
- **Código e:** No es de bloque, no singular, no instantáneo (s3 '110' es prefijo de s4 '1101') y no UD (la secuencia '11010' admite s3 s2 o s4 s1).
- **Código f:** No es de bloque, no singular, no instantáneo (s1 '0' es prefijo de s6 '001') y no UD (la secuencia '00100' admite s1 s1 s2 o s6 s1 s1).

b. Longitud promedio L de los códigos UD

Fórmula general: $L = \sum (p_i \cdot l_i)$ para i = 1..6, con p(s1) = 1/2, p(s2) = 1/4 y p(s3..s6) = 1/16.

- **Código a:** $L = (1/2)\cdot 3 + (1/4)\cdot 3 + 4\cdot(1/16)\cdot 3 = 3.0000$ bits/símbolo.
- **Código b:** $L = (1/2)\cdot 1 + (1/4)\cdot 2 + (1/16)\cdot 3 + (1/16)\cdot 4 + (1/16)\cdot 5 + (1/16)\cdot 6 = 2.1250$ bits/símbolo.
- **Código c:** $L = (1/2)\cdot 1 + (1/4)\cdot 2 + (1/16)\cdot 3 + (1/16)\cdot 4 + (1/16)\cdot 5 + (1/16)\cdot 6 = 2.1250$ bits/símbolo.

c. Fallas y modificación mínima para los códigos no UD

- **Código d:** Existe conflicto entre s3 ('110') y s6 ('1101'). Modificación mínima: reemplazar s6 por '1100'.
- **Código e:** Existe conflicto entre s3 ('110') y s4 ('1101'). Modificación mínima: reemplazar s4 por '1100'.
- **Código f:** Existe conflicto entre s1 ('0') y s6 ('001'). Modificación mínima: reemplazar s6 por '1010'.

Ejercicio 2

a. Validez del conjunto de longitudes (base trinaria D = 3)

Criterio de Kraft-McMillan: $K = \sum (3^{(-l_i)}) \leq 1$.

- **Código A:** $K = 2\cdot 3^{(-1)} + 1\cdot 3^{(-2)} + 2\cdot 3^{(-3)} + 1\cdot 3^{(-4)} + 4\cdot 3^{(-5)} = 214 / 243 \approx 0.8807 \leq 1$. Código válido.

- **Código B:** K = 2·3^(-1) + 2·3^(-2) + 2·3^(-3) + 3·3^(-4) + 1·3^(-5) = 244 / 243 ≈ 1.0041 > 1. Código no válido.
- **Código C:** K = 1·3^(-1) + 4·3^(-2) + 6·3^(-3) = 27 / 27 = 1.0000 ≤ 1. Código válido (completo).
- **Código D:** K = 2·3^(-1) + 2·3^(-2) + 2·3^(-3) + 2·3^(-4) + 3·3^(-5) = 243 / 243 = 1.0000 ≤ 1. Código válido (completo).

b. Construcción de códigos instantáneos trinarios sobre {0, 1, 2}

- **Código A:** Longitud 1: 0, 1 | Longitud 2: 20 | Longitud 3: 210, 211 | Longitud 4: 2120 | Longitud 5: 21210, 21211, 21212, 21220.
- **Código C:** Longitud 1: 0 | Longitud 2: 10, 11, 12, 20 | Longitud 3: 210, 211, 212, 220, 221, 222.
- **Código D:** Longitud 1: 0, 1 | Longitud 2: 20, 21 | Longitud 3: 220, 221 | Longitud 4: 2220, 2221 | Longitud 5: 22220, 22221, 22222.

Ejercicio 3

Un código coma sobre un alfabeto de tamaño D utiliza un símbolo fijo (por ejemplo '0') como delimitador precedido por ráfagas del otro símbolo ('1').

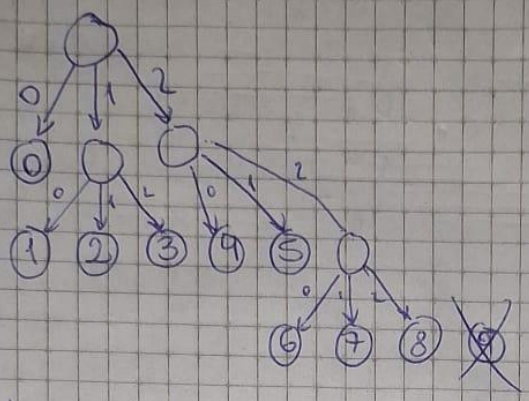
Para un conjunto finito de N símbolos sobre un alfabeto binario (D = 2), las longitudes asociadas son l_1 = 1 ('0'), l_2 = 2 ('10'), l_3 = 3 ('110'), ..., l_(N-1) = N-1 ('11..10'), l_N = N-1 ('11..11').

La sumatoria de Kraft resulta: K = ∑_(i=1)^(N-1) 2^(-i) + 2^(-(N-1)) = (1 - 2^(-(N-1))) + 2^(-(N-1)) = 1.

Para el caso infinito (N → ∞) sobre alfabeto base D: K = ∑_(i=1)^(∞) D^(-i) = (1/D) / (1 - 1/D) = 1 / (D - 1). Para D = 2, K = 1 / (2 - 1) = 1 ≤ 1.

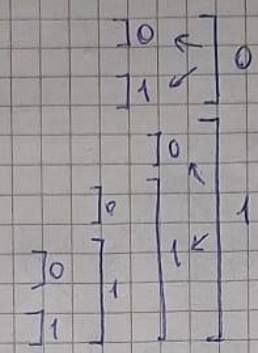
Conclusión: El código coma cumple la incondición de Kraft con igualdad estricta (K = 1).

4-


0
1
2
0
1
2
0
1
2

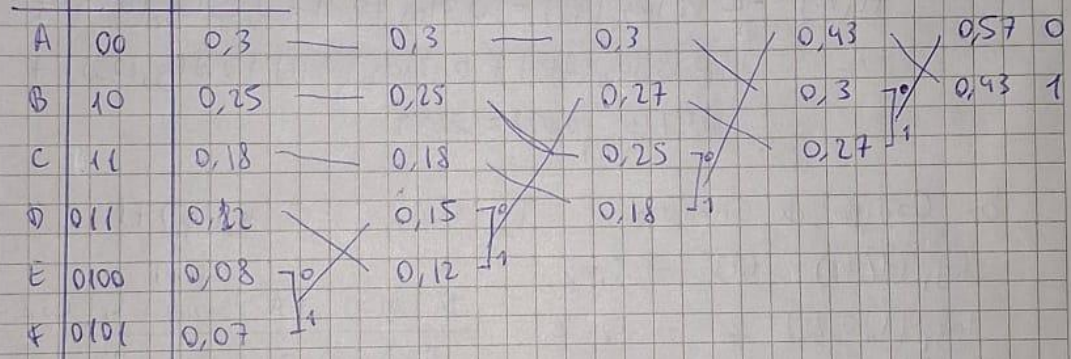
Código de Fano

Cód.	S	Prob
00	A	0,3
01	B	0,25
10	C	0,18
110	D	0,12
1110	E	0,08
1111	F	0,07



Código de Huffman

Si	Cód.	Prob.
A	00	0,3
B	10	0,25
C	11	0,18
D	011	0,12
E	0100	0,08
F	0101	0,07



b. Longitud promedio $L = \sum p_i \cdot l_i$

$L_S = 2,45$ $L_F = 2,42$ $L_H = 2,42$

Eficiencia: $\eta = H(X)/L$ Redundancia: $R = 1 - \eta$

$\eta_S = 2,39/2,45 = 0,9755$ $R_S = 0,0245$

$\eta_F = \frac{2,39}{2,42} = 0,9876$ $R_F = 0,01239$

$\eta_H = \frac{2,39}{2,42} = 0,9876$ $R_H = 0,01239$

c- Fano y Huffman dan los mismos resultados, ambos se aproximan al límite teórico mayor que Shannon.

6 - Codificación LZ



Archivo Comprimido

ERRECO1N011 001 051 031 1C 1I 1G 1A 001 051 1O 1E 022 031 1C 001 1N
E R R E R O

021 001 051 031 1B 1A 012 1I 1L
E R R E RR

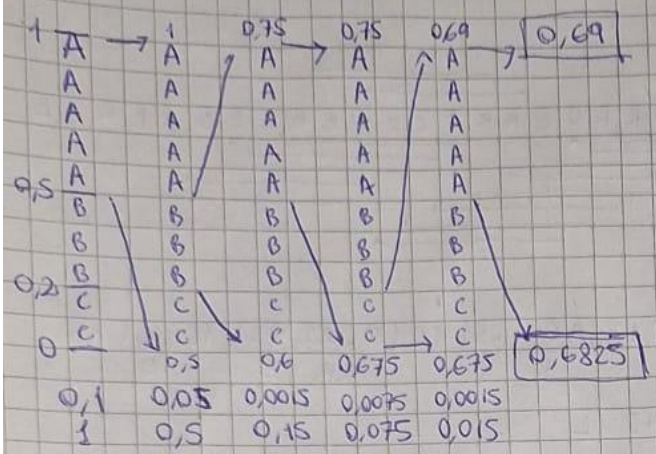
Coincidencia: 0 No coincidencia: 1 (1er bit)

2do dígito indica la posición dentro de la ventana (3 bits)

3er dígito indica la cantidad de símbolos que coincidieron, limitado por la ventana de anticipación de 4 símbolos (2 bits)

Ej: la coincidencia de la O (001) queda en binario como 000001

7- Codificación Aritmética ABACA



0,69	×2	
1,38	1	0,5
0,38		
0,76	0	0,5
1,52	1	0,625
0,52		
1,04	1	0,6875 > 0,6825
~~0,04~~		

Código binario 1011

b- Pasaje a decimal 1011

$= 2^{-1} + 2^{-3} + 2^{-4} = 0,5 + 0,125 + 0,0625 = 0,6875$

1 A
0,9 A → 0,833 5°
0,8 A
0,7 A → 0,6875 1°
0,6 A → 0,583 3°
0,5 B
0,4 B → 0,375 2°
0,3 B
0,2 C → 0,166 4°
0,1 C
0 C

Fórmula: $v = \frac{V-L}{H-L}$ $L_{nuevo} = L + (H-L) \times F(S_{i-1})$ $H_{nuevo} = L + (H-L) \times F(S_i)$

V = 0,6875
L límite inferior
H límite superior

$v = \frac{0,6875-0,5}{1-0,5} = 0,375$

$L_{nuevo} = 0,5 + 0,5 \times 0,2 = 0,6$
$H_{nuevo} = 0,5 + 0,5 \times 0,5 = 0,75$
$v = \frac{0,6875-0,6}{0,75-0,6} = 0,583$

$L_{nuevo} = 0,6 + (0,75-0,6) \times 0,5 = 0,675$
$H_{nuevo} = 0,6 + (0,75-0,6) \times 1 = 0,75$
$v = \frac{0,6875-0,675}{0,75-0,675} = 0,166$

Texto en claro: ABACA

v	L	H	Símbolo
0,6875	0	1	A
0,375	0,5	1	B
0,583	0,6	0,75	A
0,166	0,675	0,75	C
0,833	0,675	0,69	A

FUENTES DE MARKOV

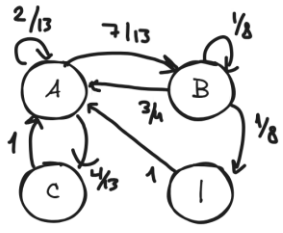
Transiciones y Probabilidades Condicionales:

- Desde A (13): -> A (2), -> B (7), -> C (4). Probabilidades: P(A|A) = 2/13, P(B|A) = 7/13, P(C|A) = 4/13.
- Desde B (8): -> A (6), -> B (1), -> | (1). Probabilidades: P(A|B) = 6/8 = 3/4, P(B|B) = 1/8, P(| | B) = 1/8.
- Desde C (3): -> A (3). Probabilidades: P(A|C) = 3/3 = 1.
- Desde | (1): -> A (1). Probabilidades: P(A| |) = 1/1 = 1.

Matriz de Transición:

Estado	**A**	**B**	**C**	**|**
A	2/13	7/13	4/13	0
B	3/4	1/8	0	1/8
C	1	0	0	0
|	1	0	0	0

Grafo de Transición:



2. Análisis a partir del ejercicio 1

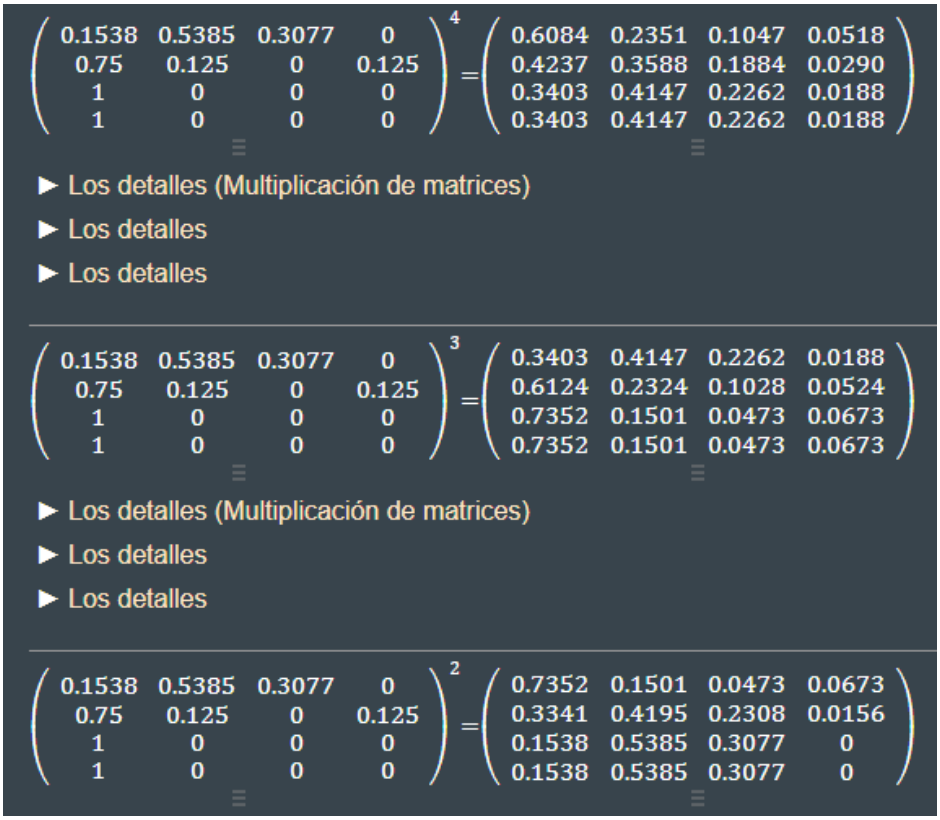
a. Matriz estacionaria: Resolviendo el sistema de ecuaciones de estado P_est = P_est * Matriz, junto con la suma de probabilidades = 1:

1. P(A) = (2/13)P(A) + (3/4)P(B) + P(C) + P(|)
2. P(B) = (7/13)P(A) + (1/8)P(B) => (7/8)P(B) = (7/13)P(A) => P(B) = (8/13)P(A)
3. P(C) = (4/13)P(A)
4. P(|) = (1/8)P(B) => P(|) = (1/13)P(A)

Sustituyendo en la sumatoria: P(A) + (8/13)P(A) + (4/13)P(A) + (1/13)P(A) = 1 => 2 * P(A) = 1 => P(A) = 0.5.

Distribucion estacionaria: P(A) = 0.5, P(B) = 0.3077, P(C) = 0.1538, P(|) = 0.0385.

b. Menor variabilidad en potencias sucesivas: Al elevar la matriz de transicion a potencias 2, 3, etc., los valores de cada columna convergen gradualmente. Para una potencia suficientemente grande, las filas se vuelven identicas, reflejando las probabilidades independientes de los estados, reduciendo la variabilidad a cero.



c. Distribucion estacionaria en el cuadrado de la matriz:

Matematicamente, si el vector de distribucion es P_est, se cumple P_est * Matriz = P_est. Multiplicando por la Matriz nuevamente, P_est * (Matriz al cuadrado) = (P_est * Matriz) * Matriz = P_est * Matriz = P_est. La distribucion se mantiene inalterada.

d. Proceso ergodico: Si. Es un proceso ergodico porque es irreducible (todos los estados se comunican entre si y pueden ser alcanzados desde cualquier otro en un numero finito de pasos) y aperiodico (el estado A tiene probabilidad de auto-transicion P(A|A) > 0).

3. En una matriz estocastica P, la suma de los elementos de cada fila siempre es 1. Esto significa que si restamos la matriz identidad (P - I), la suma de las columnas da cero. Esta ecuación demuestra que el determinante de (P - I) es cero, lo que implica directamente que las columnas (y por transposicion, las filas) de (P - I) son linealmente dependientes. Esta propiedad estructural es la que garantiza la existencia de al menos una distribucion estacionaria en el proceso de Markov.

4. a. El sistema es un modelo de Markov de orden 3, ya que la probabilidad del estado actual depende estrictamente de los tres estados inmediatamente anteriores.

b. El sistema puede encontrarse en 4 al cubo = 64 estados posibles definidos por las tripletas de simbolos.

c. Para encontrar las probabilidades independientes p(si), el sistema de ecuaciones adopta la forma estacionaria sobre el espacio de los 64 estados. Se debe plantear la ecuacion de equilibrio para todas las transiciones de las tripletas, resolver el sistema completo considerando que la suma de todas las probabilidades es 1, y finalmente sumar las probabilidades de todas las tripletas que terminan en si.

5. a. Matriz de transicion:

   Estados: 1 = Anda, 2 = No anda.

Estado	**1**	**2**
1	0.75	0.25
2	1.00	0

b. Potencias sucesivas:

Potencia 1: [0.75, 0.25 ; 1.00, 0.00]

Potencia 2: [0.8125, 0.1875 ; 0.75, 0.25]

Potencia 3: [0.7968, 0.2031 ; 0.8125, 0.1875]

Potencia 4: [0.8007, 0.1992 ; 0.7968, 0.2031]

Potencia 5: [0.7998, 0.2001 ; 0.8007, 0.1992]

Analisis: Al llegar a la quinta potencia, los valores oscilan levemente alrededor de 0.8 y 0.2, demostrando una rapida convergencia hacia la estabilidad independientemente del estado de partida.

c. Matriz de equilibrio:

Se resuelve P(1) = 0.75*P(1)* + *1.0*P(2) y P(1) + P(2) = 1.

0.25*P(1)* = *P(2)* => *0.25*P(1) = 1 - P(1) => 1.25*P(1) = 1 => P(1) = 0.8.

La distribucion estacionaria es P(1) = 0.8, P(2) = 0.2. La matriz de equilibrio es una matriz de 2x2 donde ambas filas son [0.8, 0.2].

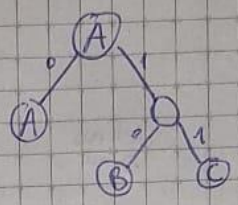
6. Longitud de codigo compacto para fuente de Markov Calculo de las probabilidades independientes (estacionarias) resolviendo el sistema para la matriz dada: P(a) = (3/4)P(a) + (1/4)P(b) + (1/4)P(c) P(b) = (1/8)P(a) + (1/2)P(b) + (1/4)P(c) P(c) = (1/8)P(a) + (1/4)P(b) + (1/2)P(c) Resulta en P(a) = 0.5, P(b) = 0.25, P(c) = 0.25.

Codificacion Independiente (L): Asignando Huffman a las probabilidades independientes (0.5, 0.25, 0.25): l(a)=1, l(b)=2, l(c)=2. L = 0.5(1) + 0.25(2) + 0.25(2) = 0.5 + 0.5 + 0.5 = 1.5 = 12/8 bits/simbolo.

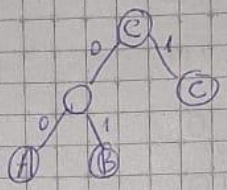
Codificacion de Markov (LM): Se crea un codigo para cada estado segun sus transiciones: Desde A (0.75, 0.125, 0.125): l(a)=1, l(b)=2, l(c)=2 => La = 0.75(1) + 0.125(2) + 0.125(2) = 1.25 = 10/8. Desde B (0.25, 0.5, 0.25): l(a)=2, l(b)=1, l(c)=2 => Lb = 0.25(2) + 0.5(1) + 0.25(2) = 1.5 = 12/8. Desde C (0.25, 0.25, 0.5): l(a)=2, l(b)=2, l(c)=1 => Lc = 0.25(2) + 0.25(2) + 0.5(1) = 1.5 = 12/8. LM = P(a)*La* + *P(b)Lb* + *P(c)Lc* = *0.5(10/8)* + *0.25(12/8)* + *0.25*(12/8) = 5/8 + 3/8 + 3/8 = 11/8. Se comprueba que LM = 11/8 y L = 12/8, cumpliendose que LM es menor que L.

7- Codificación Huffman como fuente de Markov de orden 1.

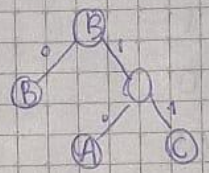
A→A 3/4 — 3/4 0
A→B 1/8 — 1/4 1
A→C 1/8



C→A 1/4 — 1/2 0
C→B 1/4 — 1/2 1
C→C 1/2



B→B 1/2 — 1/2 0
B→A 1/4 — 1/2 1
B→C 1/4



Codificación Fano como fuente de Markov de orden 1.

A→A	3/4	0
A→B	1/8	10
A→C	1/8	11

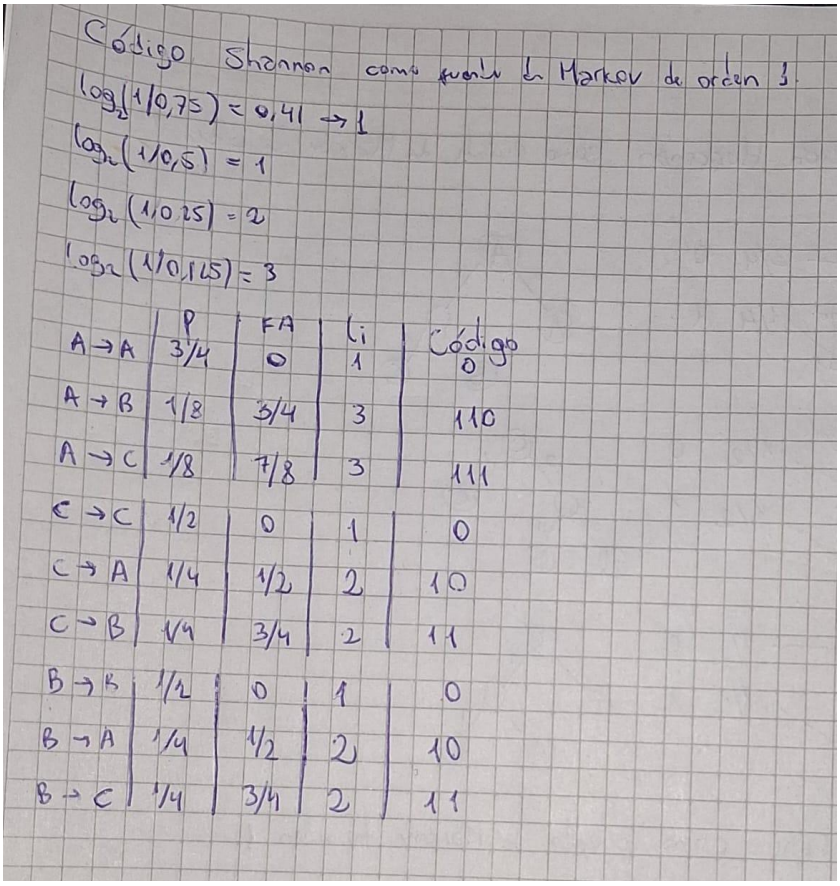
] 3/4 0
] 0] 2/8 1
] 1]

C→C	1/2	0
C→A	1/4	10
C→B	1/4	11

] 1/2 0
] 0] 2/4 1
] 1]

B→B	1/2	0
B→A	1/4	10
B→C	1/4	11

] 1/2 0
] 0] 2/4 1
] 1]



Código Shannon como fuente de Markov de orden 1.

$\log_2(1/0{,}75) = 0{,}41 \rightarrow 1$

$\log_2(1/0{,}5) = 1$

$\log_2(1/0{,}25) = 2$

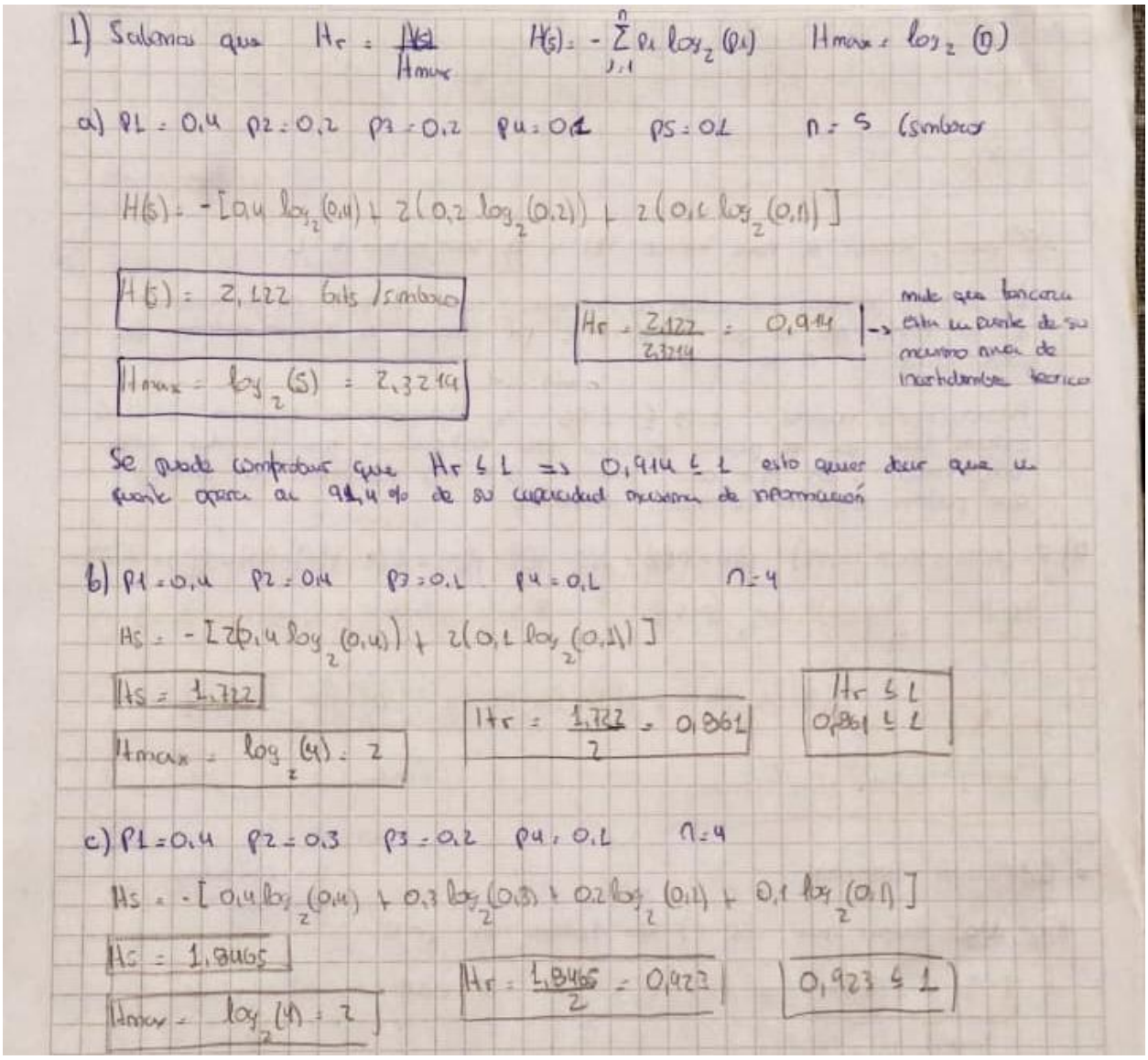
$\log_2(1/0{,}125) = 3$

	P	FA	l_i	Código
A → A	3/4	0	1	0
A → B	1/8	3/4	3	110
A → C	1/8	7/8	3	111
C → C	1/2	0	1	0
C → A	1/4	1/2	2	10
C → B	1/4	3/4	2	11
B → B	1/2	0	1	0
B → A	1/4	1/2	2	10
B → C	1/4	3/4	2	11

b. Estadistica de primer orden y codificacion predictiva: La estadistica de primer orden asume independencia absoluta, calculando el peso total: P(A)=0.5, P(B)=0.31, P(C)=0.15. La codificacion predictiva explota la probabilidad de acierto entre el simbolo esperado (basado en el historial) y el que realmente aparece. Un predictor evaluaria la transicion mas probable (por ejemplo, siempre predecir B despues de A) y, si acierta, emitiria una cadena de ceros equivalente a los aciertos consecutivos, lo que reduce la longitud requerida frente al envio del simbolo en si.

c. BWT y RLE: La Transformada de Burrows-Wheeler (BWT) reorganiza alfabeticamente todas las rotaciones ciclicas del texto AABABACABABACABBABAACAB|AC y extrae la ultima columna. Esto no comprime el texto por si mismo, pero agrupa estadisticamente los caracteres que comparten contextos similares (generando corridas de letras A o B juntas). Una vez transformada, se aplica RLE (Run-Length Encoding) que sustituye bloques de caracteres identicos por el caracter seguido de su conteo (por ejemplo, A5B3), lo cual es sumamente eficiente tras el agrupamiento de la BWT.

Práctico 4



1) Sabemos que $H_r = \frac{H(s)}{H_{max}}$ $\quad H(s) = -\sum_{i=1}^{n} p_i \log_2(p_i)$ $\quad H_{max} = \log_2(n)$

a) $p_1 = 0,4$ $p_2 = 0,2$ $p_3 = 0,2$ $p_4 = 0,1$ $p_5 = 0,1$ $\quad n = 5$ (símbolos)

$H(s) = -[0,4 \log_2(0,4) + 2(0,2 \log_2(0,2)) + 2(0,1 \log_2(0,1))]$

$H(s) = 2,122$ bits/símbolo

$H_{max} = \log_2(5) = 2,3219$

$H_r = \frac{2,122}{2,3219} = 0,914$ → más que bancarse está en puede de su máximo nivel de incertidumbre teórico

Se puede comprobar que $H_r \leq 1 \Rightarrow 0,914 \leq 1$ esto quiere decir que la fuente opera al 91,4 % de su capacidad máxima de información

b) $p_1 = 0,4$ $p_2 = 0,4$ $p_3 = 0,1$ $p_4 = 0,1$ $\quad n = 4$

$H_s = -[2(0,4 \log_2(0,4)) + 2(0,1 \log_2(0,1))]$

$H_s = 1,722$

$H_{max} = \log_2(4) = 2$

$H_r = \frac{1,722}{2} = 0,861$

$H_r \leq 1$
$0,861 \leq 1$

c) $p_1 = 0,4$ $p_2 = 0,3$ $p_3 = 0,2$ $p_4 = 0,1$ $\quad n = 4$

$H_s = -[0,4 \log_2(0,4) + 0,3 \log_2(0,3) + 0,2 \log_2(0,2) + 0,1 \log_2(0,1)]$

$H_s = 1,8465$

$H_{max} = \log_2(4) = 2$

$H_r = \frac{1,8465}{2} = 0,923$

$0,923 \leq 1$

2) $P(S1) = P(S2) = P(S3) = P(S4) = 1/4$

$H(S) = -[4(0,25 \log_2 0,25)]$

$\boxed{H(S) = 2 \frac{\text{bits}}{\text{simb}}}$

Sabemos que $H(S) \leq L$ entonces comprobando valores $2 \leq 1,86$. Sabemos que la entropía constituye el límite inferior (cota inferior) teórico absoluto de información de la fuente por lo tanto el valor exacto que puede tomar L es 2 bits/símbolo al proponer un valor de 1,86 podemos decir que no cumple con este requisito lo que se traduce en que todo intento de comprimir por debajo de este límite resulta en pérdida de información por lo que no será posible encontrar una codificación para $L: 1,86$

3) $P(S1) = (1/2)$ $P(S2) = (1/4)$ y $P(S3) = P(S4) = 1/8$

- 1er paso: cálculo a límite teórico $H(S)$

$H(S) = -[0,5 \log_2(0,5) + 0,25 \log_2(0,25) + 2(0,125 \log_2(0,125))]$

$\boxed{H(S) = 1,75 \text{ bits/símbolo}}$

- 2do paso: Acercarse a límite teórico $H(S)$ y el valor mínimo de L

La longitud promedio sin pérdida de información está limitada por la entropía de la fuente por lo que $H(S) \leq L$ por lo tanto el valor mínimo que puede tomar L es de 1,75.

Reemplazando valores $1,75 \leq 1,86$ y sabiendo la condición $L: 1,86$ podemos decir que es posible encontrar una codificación que promedie 1,86 bits/símbolo. En este caso sobrarían 0.11 bits adicionales por símbolo que podrían representar información redundante

4) $S = (s1, s2, s3, s4, s5, s6, s7)$ $P(1) = P(2) = 1/3$ $P(3) = P(4) = 1/9$ $P(5) = P(6) = P(7) = 1/27$

$$H_3(S) = -\sum_{i=1}^{7} p(s_i) \cdot \log_3 (p(s_i))$$ * cada probabilidad es potencia de 3

$$= -\left[2(1/3 \log_3 (1/3)) + 2(1/9 \log_3 (1/9)) + 3(1/27 \log_3 (1/27)) \right]$$

$$H_3(S) = 1,444 \text{ tribits / símbolo}$$

- Codificación Huffman

q = cant símbolos
r = base

1ro) Se debe cumplir que $(q - r)$ sea divisible en $(r-1)$

$\underbrace{7-3 \;/\; 3-1}_{(\text{mod } 2) = 0} = 4/2 = 2$ es entero no hacen falta símbolos ficticios

2do) Ordeno las probabilidades de mayor a menor y armo el árbol

S	cod	probabilidad					
S1	0	1/3	—	1/3	—	1/3	0
S2	1	1/3	—	1/3	—	1/3	1
S3	20	1/9	—	1/9	0		
S4	210	1/9	—	1/9	1	1/3	2
S5	220	1/27	0				
S6	221	1/27	1	1/9	2		
S7	222	1/27	2				

3ro) Reordeno en caso de ser necesario

4to) Asigno numeral

5to) Calculo la longitud promedio

$$L = \sum_{i=7}^{7} p(s_i) \cdot l(s_i)$$

$L = 2((1/3)\cdot 1) + 2(2/9) + 3((1/27)\cdot 3)$

$L = \frac{13}{9} \cong 1,444$ símbolos/símbolo

Podemos concluir que $L = H_2(S)$ por lo que el algoritmo según Huffman genera un código 100 % eficiente, compacto y sin redundancias

5) $P(S1) = 1/2 \quad P(S2) = P(S3) = 1/4$

$H(S) = -[1/2 \log_2(1/2) + 2(1/4 \log_2(1/4))]$

$H(S) = 1,5$ bits/símbolo

- Codificación de Huffman

S	código	probabilidad	
S1	0	0,5	0,5 — 0
S2	10	0,25	0,5 — 1
S3	11	0,25	

$L_H = \sum P(S_i) \cdot l(S_i) = (1/2 \cdot 1) + (1/4 \cdot 2) + (1/4 \cdot 2) = 1,5$ bits

$L_H = H_S$ ∴ la codificación es 100 % eficiente compacta y sin redundancias

- Codificación de Shannon

1ro) Ordeno de mayor a menor las probabilidades 2do) Calculo $l_i = \log_2(1/P_i)$ y l_i

S	P(Si)	FA	FA binario	li	código
S1	0,5	0	0,00	1	0
S2	0,25	0,5	0,10	2	10
S3	0,25	0,75	0,11	2	11

$\log_2(1/0,5) = 1$
$\log_2(1/0,5) = 2$
$\log_2(1/0,25) = 2$

3ro) Calculo la función acumulativa (FA): es la suma de las probabilidades de los símbolos anteriores de la lista. La primera siempre es FA = 0

4to) Calculo FA en binario

5to) Para la codificación nos quedamos con los l_i dígitos

6to) Calculo la longitud media y eficiencia

$L_S = (1/2 \cdot 1) + 2(1/4 \cdot 2) = 1,5$ bits

$n = H/L = \frac{1,5}{1,5} = 1 \times 100 = 100\%$ eficiencia

- Código de Fano

Pasos para codificar Fano

1ro) ordenar de mayor a menor la probabilidad

2do) dividir la tabla en 2 grupos de suma lo más parecido posible

3ro) asignar un dígito a cada grupo

4to) repetir todos de ciclo que haya + 1 símbolo

5to) Calcular L_F y eficiencia

S	p	cod
S_1	1/2	0
S_2	1/4	10
S_3	1/4	11

0 → 0
1 → 1

$L_F = (1/2 \cdot 1) + 2(1/4 \cdot 2) = 1,5$ bits

$H(S) = L_F$ ∴ la codificación es 100 % eficaz y compacta

Conclusión: se verifica que $L_H \leq L_F \leq L_S$ (todas iguales en este caso)

como $L = H_S$ $\eta = 100\%$ y la redundancia es cero

En este caso las 3 metodos coinciden ya que no han redondeado que haya podido eficacia

6) Segunda extensión

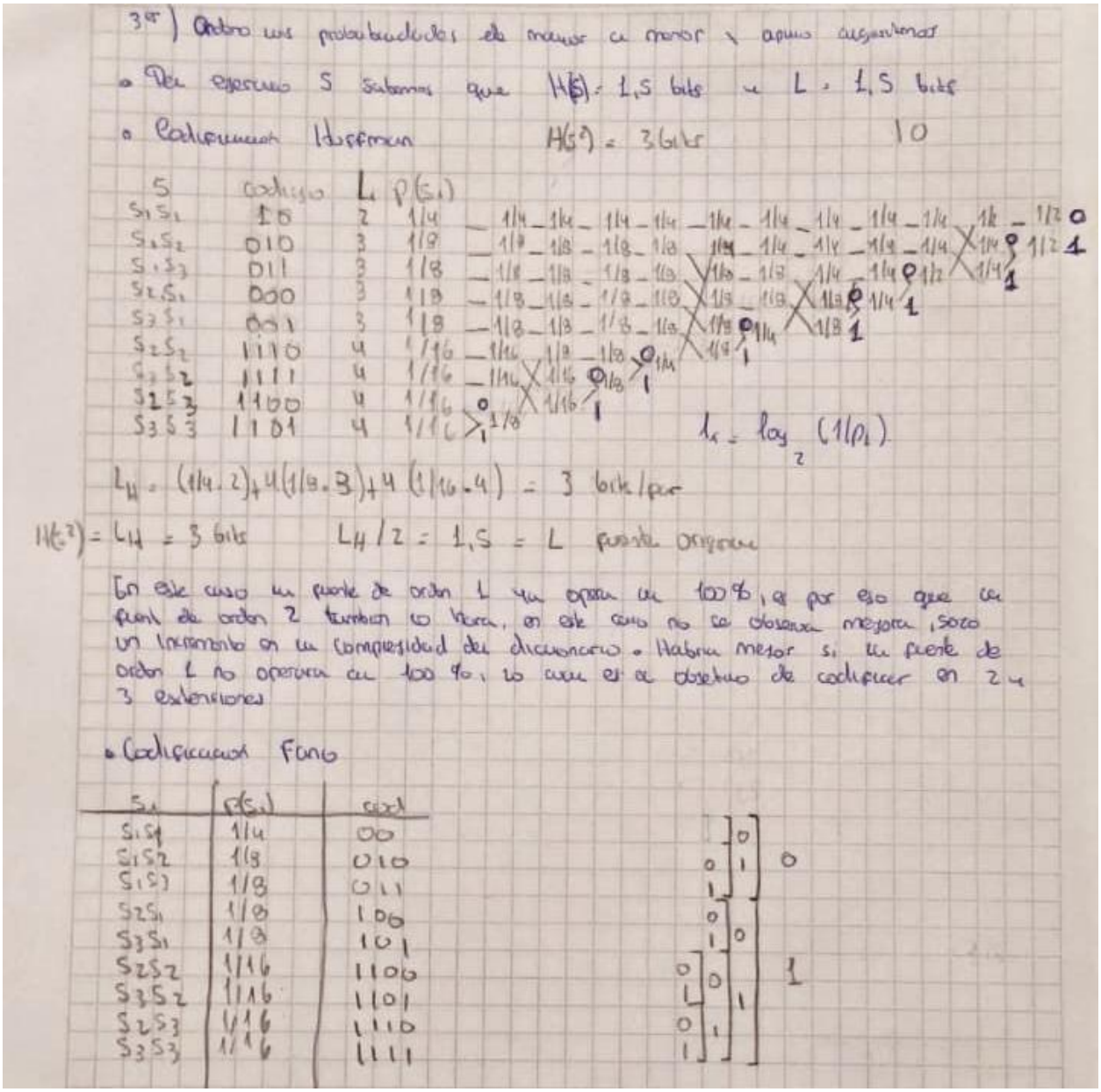
$q = 3^2 = 3^2 = 9$ símbolos para (símbolos)

1er Paso: Armo la tabla de pares

	S_1	S_2	S_3
S_1	S_1S_1	S_1S_2	S_1S_3
S_2	S_2S_1	S_2S_2	S_2S_3
S_3	S_3S_1	S_3S_2	S_3S_3

2do Paso: Calculo de las probabilidades

	1/2	1/4	1/4
1/2	1/4	1/8	1/8
1/4	1/8	1/16	1/16
1/4	1/8	1/16	1/16



Tercera extensión

Cantidad de s_1	Símbolos	p
3	$s_1s_1s_1$	1/8
2	$s_1s_1s_2$, $s_1s_1s_3$, $s_1s_2s_1$, $s_1s_3s_1$, $s_2s_1s_1$, $s_3s_1s_1$ (6 símbolos)	1/16

Cantidad de s_1	Símbolos	p
1	$s_1s_2s_2$, $s_1s_2s_3$, $s_1s_3s_2$, $s_1s_3s_3$, $s_2s_1s_2$, $s_2s_1s_3$, $s_3s_1s_2$, $s_3s_1s_3$, $s_2s_2s_1$, $s_2s_3s_1$, $s_3s_2s_1$, $s_3s_3s_1$ (12 símbolos)	1/32
0	$s_2s_2s_2$, $s_2s_2s_3$, $s_2s_3s_2$, $s_2s_3s_3$, $s_3s_2s_2$, $s_3s_2s_3$, $s_3s_3s_2$, $s_3s_3s_3$ (8 símbolos)	1/64

Longitud de los símbolos según las probabilidades

p	Longitud
1/8	3
1/16	4
1/32	5
1/64	6

Arbol de Huffman

raíz (64)
├─ 0 (32)
│ ├─ 00 (16)
│ │ ├─ 000 (8)
│ │ │ ├─ 0000 (4)
│ │ │ │ ├─ 00000 (2): 000000 $s_2s_2s_2$ · 000001 $s_2s_2s_3$
│ │ │ │ └─ 00001 (2): 000010 $s_2s_3s_2$ · 000011 $s_2s_3s_3$
│ │ │ └─ 0001 (4)
│ │ │   ├─ 00010 (2): 000100 $s_3s_2s_2$ · 000101 $s_3s_2s_3$
│ │ │   └─ 00011 (2): 000110 $s_3s_3s_2$ · 000111 $s_3s_3s_3$
│ │ └─ 001 (8)
│ │   ├─ 0010 (4): 00100 $s_1s_2s_2$ · 00101 $s_1s_2s_3$

```
│   │       └─ 0011 (4): 00110 s1s3s2 · 00111 s1s3s3
│   └─ 01 (16)
│       ├─ 010 (8)
│       │   ├─ 0100 (4): 01000 s2s1s2 · 01001 s2s1s3
│       │   └─ 0101 (4): 01010 s3s1s2 · 01011 s3s1s3
│       └─ 011 (8)
│           ├─ 0110 (4): 01100 s2s2s1 · 01101 s2s3s1
│           └─ 0111 (4): 01110 s3s2s1 · 01111 s3s3s1
└─ 1 (32)
    ├─ 10 (16)
    │   ├─ 100 (8): 1000 s1s1s2 · 1001 s1s1s3
    │   └─ 101 (8): 1010 s1s2s1 · 1011 s1s3s1
    └─ 11 (16)
        ├─ 110 (8): 1100 s2s1s1 · 1101 s3s1s1
        └─ 111 ..... s1s1s1
```

Árbol de Fano

```
raíz (64)
├─ 0 (32)
│   ├─ 00 (16)
│   │   ├─ 000 ..... s1s1s1
│   │   └─ 001 (8): 0010 s1s1s2 · 0011 s1s1s3
│   └─ 01 (16)
│       ├─ 010 (8): 0100 s1s2s1 · 0101 s1s3s1
│       └─ 011 (8): 0110 s2s1s1 · 0111 s3s1s1
└─ 1 (32)
    ├─ 10 (16)
    │   ├─ 100 (8)
    │   │   ├─ 1000 (4): 10000 s1s2s2 · 10001 s1s2s3
    │   │   └─ 1001 (4): 10010 s1s3s2 · 10011 s1s3s3
    │   └─ 101 (8)
    │       ├─ 1010 (4): 10100 s2s1s2 · 10101 s2s1s3
    │       └─ 1011 (4): 10110 s3s1s2 · 10111 s3s1s3
    └─ 11 (16)
        ├─ 110 (8)
```

- 1100 (4): 11000 $s_2s_2s_1$ · 11001 $s_2s_3s_1$
- 1101 (4): 11010 $s_3s_2s_1$ · 11011 $s_3s_3s_1$

111 (8)

- 1110 (4)
  - 11100 (2): 111000 $s_2s_2s_2$ · 111001 $s_2s_2s_3$
  - 11101 (2): 111010 $s_2s_3s_2$ · 111011 $s_2s_3s_3$
- 1111 (4)
  - 11110 (2): 111100 $s_3s_2s_2$ · 111101 $s_3s_2s_3$
  - 11111 (2): 111110 $s_3s_3s_2$ · 111111 $s_3s_3s_3$

Codificación de Huffman y Fano

Bloque	**p**	**Huffman**	**Fano**	**l_i**
$s_1s_1s_1$	1/8	111	000	3
$s_1s_1s_2$	1/16	1000	0010	4
$s_1s_1s_3$	1/16	1001	0011	4
$s_1s_2s_1$	1/16	1010	0100	4
$s_1s_3s_1$	1/16	1011	0101	4
$s_2s_1s_1$	1/16	1100	0110	4
$s_3s_1s_1$	1/16	1101	0111	4
$s_1s_2s_2$	1/32	00100	10000	5
$s_1s_2s_3$	1/32	00101	10001	5
$s_1s_3s_2$	1/32	00110	10010	5
$s_1s_3s_3$	1/32	00111	10011	5

Bloque	**p**	**Huffman**	**Fano**	**l_i**
$s_2s_1s_2$	1/32	01000	10100	5
$s_2s_1s_3$	1/32	01001	10101	5
$s_3s_1s_2$	1/32	01010	10110	5
$s_3s_1s_3$	1/32	01011	10111	5
$s_2s_2s_1$	1/32	01100	11000	5
$s_2s_3s_1$	1/32	01101	11001	5
$s_3s_2s_1$	1/32	01110	11010	5
$s_3s_3s_1$	1/32	01111	11011	5
$s_2s_2s_2$	1/64	000000	111000	6
$s_2s_2s_3$	1/64	000001	111001	6
$s_2s_3s_2$	1/64	000010	111010	6
$s_2s_3s_3$	1/64	000011	111011	6
$s_3s_2s_2$	1/64	000100	111100	6
$s_3s_2s_3$	1/64	000101	111101	6
$s_3s_3s_2$	1/64	000110	111110	6
$s_3s_3s_3$	1/64	000111	111111	6

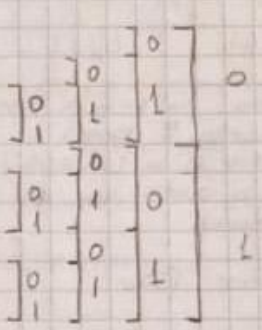
7) a) Fano → 2da extensión

PA BL IT O PA BL IT O OC LA VO 12 pares

2 2 1 1 1 1 1 1 1 1

p = 2/12 p = 1/12

S_i	p	cod
PA	1/6	00
BL	1/6	010
IT	1/12	0110
UP	1/12	0111
AB	1/12	100
LI	1/12	1010
TO	1/12	1011
OC	1/12	110
LA	1/12	1110
VO	1/12	1111



$H(S) = -\sum_{i=1}^{12} p_i \log_2(p_i)$

$= 0,712$ bits/seg

$L_F = 1$ bit/símbolo

$\eta = \frac{H(S)}{L_F} = 72,2\%$

$H(S^2) = 1,44$ bits

$L_2 = 1,56$ bits

$\eta_2 = \frac{H(S^2)}{L_2} = 92,3\%$

Podemos concluir que realizando una extensión de orden 2 la eficiencia del código incrementa significativamente

7) b) PABLO PABLO PABLO OCAVO } 40 bits son transmitidos

2 3 4 4 3 4 4 2 3 3 4 4

El tamaño cadena es 24 caracteres y tiene 9 símbolos sin repetir

$\log_2 9 = 4$ bits $24 \times 4 = 96$ caracteres

$$C_r = \left(1 - \frac{46}{96}\right) \times 100 = 58,33\ \% \rightarrow \text{porcentaje de compresión}$$

. Codificación de Markov (1er extensión) - Fano -

Estado anterior	Símbolo siguiente	cant transiciones	probabilidad condicional	codigo Fano	long. bit
P	A	3	$p(A/P) = 3/3 = 1$	0	1
A	B	3	$p(B/A) = 3/4 = 0,75$	0	1
	V	1	$p(B/V) = 1/4 = 0,25$	1	1
B	L	3	$p(L/B) = 3/3 = 1$	0	1
L	L	2	$p(L/L) = 2/4 = 0,5$	0	1
	O	1	$p(L/O) = 1/4 = 0,25$	0	2
	A	1	$p(L/A) = 1/4 = 0,25$	1	2
I	T	2	$p(I/T) = 2/2 = 1$	0	1
T	O	2	$p(T/O) = 2/2 = 1$	0	1
O	P	2	$p(O/P) = 2/3 = 0,66$	0	1
	C	1	$p(O/C) = 1/3 = 0,33$	1	1
C	L	1	$p(C/L) = 1$	0	1
V	O	1	$p(V/O) = 1$	0	1

PABLITO PABLITO PABLO CUPOO
↓ 0000000000000000010110 } 29 bits
Símbolo inicial = 46 bits

. El tamaño original era de : 40 bits

. Tamaño comprimido es de : 29 bits

$$C = \left(1 - \frac{29}{96}\right) \times 100 = 69,8\ \% \quad \text{tasa de compresión}$$

7) c) El aumento en la tasa de compresión se debe a que en la cod. de señales antes de Fano no se tiene encuenta el estado anterior, es decir una dependencia entre los caracteres asumiendo que son estadisticamente independientes entre sí

8) a) Algoritmo RLE

Cadena original → TOMATOMATOMA$

Salida alg RLG → T1O1M1A1T1O1M1A1T1O1M1A1$1

Como no existen caracteres ... RLE fracasa generando una cadena de 26 caracteres, antes tenía 13 caracteres y luego de RLE 26. Por lo tanto la expansión a la cadena es un 100%

b) Aplicación de pipeline BWT → RLE

TOMATOMATOMA$	$TOMATOMATOMA
$TOMATOMATOMA	A$TOMATOMATOM
A$TOMATOMATOM	ATOMA$TOMATOM
MA$TOMATOMATO	ATOMATOM$TOM
OMA$TOMATOMAT	MA$TOMATOMAT
TOMA$TOMATOMA	MATOMA$TOMAT
ATOMA$TOMATOM	MATOMATOMA$T
MATOMA$TOMATO	OMA$TOMATOMA
OMATOMA$TOMAT	OMATOMA$TOMA
TOMATOMA$TOMA	OMATOMATOMA$
ATOMATOMA$TOM	TOMA$TOMATOM
MATOMATOMA$TO	TOMATOMA$TOM
OMATOMATOMA$T	TOMATOMATOMA

A	M
M	M
M	M
O	T
O	T
T	A
T	A
A	$

Salida BWT: AMMOOTTAA$

- Aplicación de RLG → A1M3O3T3A2$1